

assignment-05-code

September 9, 2026

1 Assignment 05

1.1 Import External Libraries

```
[1]: import numpy as np
import sympy as sp

from IPython.display import display, Math
```

1.2 4. Colab Verification (Problems 1–3)

```
[2]: # --- Problem 4: verify Problems 1-3 in Colab ---
xs = [np.array([1, 1]) / np.sqrt(2), np.array([1, -1]) / np.sqrt(2)] # |x+>, |x->
ys = [np.array([1, 1j]) / np.sqrt(2), np.array([1, -1j]) / np.sqrt(2)] # |y+>, |y->

# Problem 1: change of basis U_ij = <x_i | y_j>
U = np.array([np.vdot(xi, yj) for yj in ys for xi in xs])
display(Math("U = " + sp.latex(sp.Matrix(np.round(U, 4)))))
print("Problem 1 -- U unitary?", np.allclose(U.conj().T @ U, np.eye(2)))

print("-----")

# Problem 2: completeness sum_i |x_i><x_i| = I
completeness = sum(np.outer(xi, xi.conj()) for xi in xs)
display(Math(r"\sum_i \ket{x_i}\bra{x_i} = " + sp.latex(sp.Matrix(np.round(completeness.real, 4)))))
print("Problem 2 -- equals I?", np.allclose(completeness, np.eye(2)))

print("-----")

# Problem 3: spectral decomposition sum_i lambda_i |x_i><x_i| = sigma_x
spectral = sum(lam * np.outer(xi, xi.conj()) for lam, xi in zip((1, -1), xs))
display(Math(r"\sum_i \lambda_i \ket{x_i}\bra{x_i} = " + sp.latex(sp.Matrix(np.round(spectral.real, 4)))))
print("Problem 3 -- equals sigma_x?", np.allclose(spectral, [[0, 1], [1, 0]]))
```

$$U = \begin{bmatrix} 0.5 + 0.5i & 0.5 - 0.5i \\ 0.5 - 0.5i & 0.5 + 0.5i \end{bmatrix}$$

Problem 1 -- U unitary? True

$$\sum_i |x_i\rangle \langle x_i| = \begin{bmatrix} 1.0 & 0.0 \\ 0.0 & 1.0 \end{bmatrix}$$

Problem 2 -- equals I? True

$$\sum_i \lambda_i |x_i\rangle \langle x_i| = \begin{bmatrix} 0.0 & 1.0 \\ 1.0 & 0.0 \end{bmatrix}$$

Problem 3 -- equals sigma_x? True

1.3 8. Tensor Products using Colab (Problems 5–7)

```
[3]: # --- Problem 8: verify Problems 5-7 with np.kron ---
f = np.array([0, 1])
e = np.array([1, 0])
g = np.array([1, 1j]) / np.sqrt(2)

# Problem 5: the two orderings differ
fg, gf = np.kron(f, g), np.kron(g, f)
display(Math(r"\ket{f}\otimes\ket{g} = " + sp.latex(sp.Matrix(np.round(fg, 4)).T)))
display(Math(r"\ket{g}\otimes\ket{f} = " + sp.latex(sp.Matrix(np.round(gf, 4)).T)))
print("Problem 5 -- |f>|g> == |g>|f> ?", np.allclose(fg, gf))

print("-----")

# Problem 6: three-fold tensor product |0> (x) |g> (x) |0>
triple = np.kron(np.kron(f, g), f)
print("Problem 6 -- dimension:", triple.shape[0])

print("-----")

# Problem 7: distributivity from both sides
print("Problem 7 -- left :", np.allclose(np.kron(f + e, g), np.kron(f, g) + np.kron(e, g)))
print("Problem 7 -- right :", np.allclose(np.kron(g, f + e), np.kron(g, f) + np.kron(g, e)))
```

$$|f\rangle \otimes |g\rangle = \begin{bmatrix} 0 & 0 & 0.7071 & 0.7071i \end{bmatrix}$$

$$|g\rangle \otimes |f\rangle = \begin{bmatrix} 0 & 0.7071 & 0 & 0.7071i \end{bmatrix}$$

Problem 5 -- |f>|g> == |g>|f> ? False

Problem 6 -- dimension: 8

Problem 7 -- left : True

Problem 7 -- right : True

1.4 11. Tensor Product Calculation using Colab (Problem 10)

```
[4]: # --- Problem 11: verify Problem 10 with np.kron ---
sx = np.array([[0, 1], [1, 0]])
f = np.array([0, 1])
g = np.array([1, 1j]) / np.sqrt(2)

lhs = np.kron(sx @ f, sx @ g) # (sx|f>) (x) (sx|g>)
rhs = np.kron(sx, sx) @ np.kron(f, g) # (sx (x) sx)(|f> (x) |g>)

display(Math(r"(\sigma_x\ket{f})\otimes(\sigma_x\ket{g}) = " + sp.latex(sp.Matrix(np.round(lhs, 4)).T)))
display(Math(r"(\sigma_x\otimes\sigma_x)(\ket{f}\otimes\ket{g}) = " + sp.latex(sp.Matrix(np.round(rhs, 4)).T)))
print("Problem 11 -- equal?", np.allclose(lhs, rhs))
```

$$(\sigma_x |f\rangle) \otimes (\sigma_x |g\rangle) = \begin{bmatrix} 0.7071i & 0.7071 & 0 & 0 \end{bmatrix}$$

$$(\sigma_x \otimes \sigma_x)(|f\rangle \otimes |g\rangle) = \begin{bmatrix} 0.7071i & 0.7071 & 0 & 0 \end{bmatrix}$$

Problem 11 -- equal? True
